

约束求解器 Z3 深度学习与实践笔记

一、求解器概述与 Z3 基础环境构建

在计算机科学与数学逻辑的交叉领域，SMT (Satisfiability Modulo Theories, 可满足性模理论) 求解器是一种用于判定逻辑公式在特定理论背景下是否可满足的强大工具。与主要处理布尔变量的 SAT 求解器不同，SMT 求解器能够理解并处理包括整数、实数、位向量、数组以及代数数据类型在内的复杂数学理论。Z3 是由微软研究院 (Microsoft Research) 开发的一款高性能定理证明器，它不仅支持上述所有理论，还提供了包括 Python、C++、Java 在内的多种编程语言接口。根据《Z3APIinPython》教程的介绍，Z3Py (Z3 的 Python 接口) 因其语法对程序员友好，成为了学习和使用 Z3 的首选方式。

二、基础算术与逻辑建模 (Introduction)

学习的第一阶段主要围绕 Z3 的基础表达能力展开。Z3 的核心工作模式是：

- a. 声明符号变量
- b. 添加约束条件
- c. 求解

1. 整数与实数理论

在 Z3 中，变量不是编程语言中存储具体数值的容器，而是数学上的符号。教程展示了如何使用 `Int('x')` 和 `Real('y')` 来声明变量。特别值得注意的是，Z3Py 对算术运算符进行了重载，使得我们可以像编写普通代数方程一样构建约束。我运行了教程中的一个经典示例：求解

$$x > 2, y < 10, x > 2, y < 10, x > 2, y < 10$$

且

$$x + 2y = 7, x + 2y = 7, x + 2y = 7$$

python 代码与运行结果：



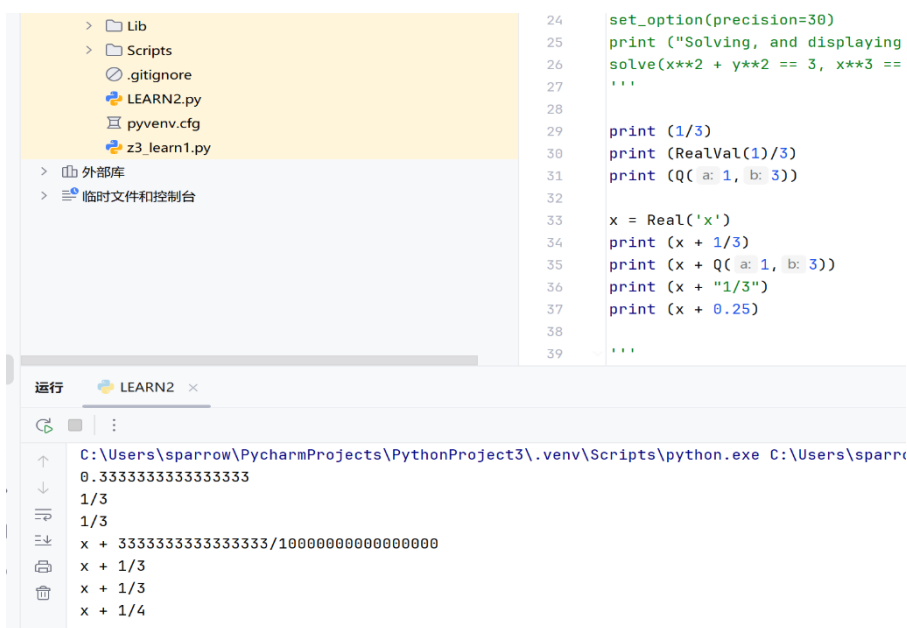
The screenshot shows the PyCharm IDE with a project named 'PythonProject3'. The file explorer on the left shows a directory structure with 'Scripts', '.gitignore', 'LEARN2.py', 'pyenv.cfg', and 'z3_learn1.py'. The main editor displays a Python script with the following code:

```
128 from z3 import *
129
130 # 声明符号整数变量
131 x = Int('x')
132 y = Int('y')
133 z = Int('z')
134 print("正在求解基础算术约束...")
135 solve(*args: x > 2, y < 10, x + 2 * y == 7)
136
137
138
139
140
141
```

The bottom pane shows the execution output for 'LEARN2.py':

```
C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\Scripts\python.exe C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\LEARN2.py
正在求解基础算术约束...
[y = 0, x = 7]
进程已结束，退出代码为 0
```

教程还特别强调了实数（Reals）的精度问题。在 Python 中， $1/3$ 通常是一个浮点数近似值，但在 Z3 的实数理论中，它是精确的分数。通过 `set_option(precision=30)`，我可以控制输出结果的小数位精度，甚至可以看到无理数（如 $1/3$ ）的精确代数表示（RootObject），这体现了 Z3 在数学严谨性上的优势。



The screenshot shows the PyCharm IDE with a project named 'PythonProject3'. The file explorer on the left shows a directory structure with 'Lib', 'Scripts', '.gitignore', 'LEARN2.py', 'pyenv.cfg', and 'z3_learn1.py'. The main editor displays a Python script with the following code:

```
24 set_option(precision=30)
25 print("Solving, and displaying
26 solve(x**2 + y**2 == 3, x**3 ==
27
28
29 print(1/3)
30 print(RealVal(1)/3)
31 print(Q(a: 1, b: 3))
32
33 x = Real('x')
34 print(x + 1/3)
35 print(x + Q(a: 1, b: 3))
36 print(x + "1/3")
37 print(x + 0.25)
38
39
```

The bottom pane shows the execution output for 'LEARN2.py':

```
C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\Scripts\python.exe C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\LEARN2.py
0.3333333333333333
1/3
1/3
x + 3333333333333333/10000000000000000
x + 1/3
x + 1/3
x + 1/4
```

2. 布尔逻辑与命题证明

Z3 不仅能解方程，还能处理布尔逻辑。通过 Bool 声明布尔变量，并结合 And, Or, Not, Implies（蕴含）等连接词，可以构建复杂的逻辑命题。

教程中演示了如何利用 Z3 证明摩根定律（DeMorgan's laws）。这里涉及到一个核心概念：有效性（Validity）与可满足性（Satisfiability）。一个公式

F 是有效的，当且仅当它的否定 $\neg F$ 是不可满足的 (Unsatisfiable)。我编写了一个 prove 函数，它通过检查否定的不可满足性来自动证明逻辑重言式。

```
p, q = Bools('p q')
# 构造摩根定律公式
demorgan = And(*args: p, q) == Not(Or(*args: Not(p), Not(q)))
print("正在证明摩根定律...")

def prove(f):
    s = Solver()
    s.add(Not(f)) # 添加否定的约束
    if s.check() == unsat:
        print("证明成功 (Proved)")
    else:
        print("证明失败 (Failed to prove)")

prove(demorgan)
```

这个实验让我理解了 SMT 求解器在自动化定理证明领域的应用逻辑。

三、求解器控制与增量求解

在掌握了基础语法后，我深入学习了 Solver 对象的使用。虽然便捷函数 solve() 能快速出结果，但 Solver 对象提供了更精细的控制能力，这对于解决复杂问题至关重要。

1. 约束堆栈与回溯 (Push&Pop)

教程详细介绍了求解器的增量求解功能。Solver 维护一个断言 (Assertions) 堆栈。push() 命令可以创建一个新的作用域 (Scope)，保存当前堆栈的大小；而 pop() 命令则可以回退到之前的状态，撤销中间添加的所有约束。

这一特性在路径搜索或尝试不同参数配置时极为有用。我模拟了一个实验：首先添加一组基础约束，然后通过 push() 进入尝试分支，添加一个导致冲突 (unsat) 的约束，检查无解后，再通过 pop() 回退并尝试另一条路径。

```
s=Solver()
x=Int('x')
```

```

s.add(x>10)

print("基础约束:x>10")

s.push()#以此为基准点

s.add(x<5)#添加冲突约束

print("尝试添加 x<5，检查结果:",s.check())#结果应为 unsat

s.pop()#回退

print("回退后再次检查:",s.check())#结果应恢复为 sat

```

这种机制避免了每次重新构建整个问题，极大地提高了求解效率。



The screenshot shows a PyCharm IDE with a Python script on the left and its execution output on the right. The script defines a solver, adds a constraint $x > 10$, prints it, pushes the state, adds a conflicting constraint $x < 5$, prints the result (unsat), pops the state, and prints the result after popping (sat). The output window shows the corresponding printed messages.

```

155 s = Solver()
156 x = Int('x')
157 s.add(x > 10)
158 print("基础约束: x > 10")
159
160 s.push() # 以此为基准点
161 s.add(x < 5) # 添加冲突约束
162 print("尝试添加 x < 5，检查结果:", s.check()) # 结果应为 unsat
163
164 s.pop() # 回退
165 print("回退后再次检查:", s.check()) # 结果应恢复为 sat
166

```

运行 LEARN2 ×

```

C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\Scripts\python.exe C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\LEARN2.py
基础约束: x > 10
尝试添加 x < 5，检查结果: unsat
回退后再次检查: sat

```

2. 结果检查与模型获取

当 `s.check()` 返回 `sat` 时，我们可以通过 `s.model()` 获取具体的解（Interpretation）。教程中提到，模型不仅给出了变量的具体值，还可以像字典一样被访问（如 `m[x]`）。如果求解结果是 `unknown`，通常意味着问题超出了当前理论的判定范围（例如非线性的多项式计算有时会无法判定）。

四、机器算术：位向量（Bit-Vectors）

Introduction 的后半部分介绍了与计算机底层密切相关的“机器算术”。现代 CPU 处理的是固定长度的整数（如 32 位或 64 位），这引入了溢出和截断的问题，普通的数学整数无法准确模拟这些行为。

1. 位向量基础

Z3 提供了 BitVec 排序。我尝试创建了一个 16 位的位向量，并验证了溢出行为。例如，对于 16 位无符号整数， $65535+1$ 应该等于 0。

```
x=BitVec('x', 16)
solve(x==65535, x+1==0)
```



此外,教程还区分了有符号运算和无符号运算。在 Z3Py 中,标准的`<`,`<=`,`>`,`>=`运算符被视为有符号比较,而 `ULT`(Unsigned Less Than)等专用函数用于无符号比较。这对于逆向分析 C 语言代码中的比较逻辑非常关键。

2. 位运算技巧验证

教程列举了一些 C 语言程序员常用的“位操作黑魔法”(BitTricks),并演示了如何用 Z3 证明其正确性。

其中一个经典的例子是判断一个数是否为 2 的幂:`x!=0&&!(x&(x-1))`。我编写了代码,让 Z3 遍历所有可能的 32 位整数情况(通过符号化验证而非穷举),证明了该表达式确实等价于“x 是 2 的幂”。

```
def prove_power_of_two():
    fast_check = And(x != 0, (x & (x - 1)) == 0)
    powers = [2 ** i for i in range(32)]
    slow_check = Or([x == p for p in powers])
    theorem = (fast_check == slow_check)
    s = Solver()
    s.add(Not(theorem)) # 添加“反例存在”的假设
    print("正在证明位运算技巧 x != 0 && !(x & (x-1)) ...")
    result = s.check()
    if result == unsat:
        print(">> 证明成功 (unsat)! ")
        print(">> 解释: 找不到任何一个 32 位整数能违反这个等式。")
        print(">> 结论: 该表达式确实严格等价于“判断是否为 2 的幂”。")
    else:
        print(">> 证明失败, 找到了反例: ", s.model())
```

```
if __name__ == "__main__":  
    prove_power_of_two()
```

运行结果：

```
C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\  
正在证明位运算技巧  $x \neq 0 \ \&\& \ ! (x \ \& \ (x-1)) \dots$   
>> 证明成功 (unsat)!  
>> 解释：找不到任何一个 32 位整数能违反这个等式。  
>> 结论：该表达式确实严格等价于“判断是否为 2 的幂”。  
  
进程已结束，退出代码为 0
```

五、Z3 进阶特性：AST 与表达式操作 (Advanced)

进入 AdvancedTopics 的学习，我开始接触 Z3 的底层表示。在 Z3 中，所有的表达式、排序 (Sorts) 和声明本质上都是抽象语法树 (AST)。

1. 表达式结构分析

通过 `x.sort()`, `x.decl()` 和 `x.children()` 等方法，我可以遍历表达式的内部结构。例如，表达式 `x+y` 在内部是一个函数应用 (Application)，其声明是加法操作符，子节点是变量 `x` 和 `y`。教程演示了如何使用 `substitute` 函数来替换表达式中的部分内容，这在进行符号化简或重写规则时非常有用。

2. 列表推导式与向量生成

为了简化大量变量的创建，Z3Py 支持 Python 的列表推导式。教程中展示了如何快速生成一个包含 100 个变量的列表，或者创建一个 3×3 的矩阵。

```
# 创建 3x3 整数矩阵  
X = [[Int(f'x_{i}_{j}') for j in range(3)] for i in range(3)]
```

结合 `pp(prettyprint)` 函数，可以清晰地打印出这些复杂的结构。这为后续处理数独 (Sudoku) 或八皇后 (EightQueens) 等矩阵类问题打下了基础。



六、数组理论 (Arrays)

数组 (Arrays) 是一个极其重要的概念。不同于编程语言中的数组，Z3 中的数组遵循 McCarthy 的 Select-Store 公理，它实际上是一个定义域和值域都无限的映射函数。

1. Select 与 Store

Select(A, i) 表示读取数组 A 在索引 i 处的值，Store(A, i, v) 表示返回一个新的数组，该数组除了索引 i 处的值为 v 外，其余位置与 A 相同。

我跟随教程模拟了一个内存写入检测的场景：

```
A=Array('A', IntSort(), IntSort())  
x=Int('x')  
#Store 操作返回新的数组状态  
A1=Store(A, 0, 10)  
#验证：读取新数组 A1 的位置 0，是否等于 10  
solve(Select(A1, 0)==10)
```

教程还介绍了 ConstArray (常数数组)，即所有位置都初始化为同一个值的数组。这在模拟内存初始化 (如 memset) 时非常方便。

七、代数数据类型 (Datatypes)

Z3 支持定义类似于 ML 或 Haskell 语言中的代数数据类型。这使得我们能够直接在求解器中对列表 (List)、树 (Tree) 或枚举 (Enum) 进行建模，而不需要手动将其编码为整数。

1. 列表类型的定义

我按照教程的步骤，声明了一个递归的整数列表类型 List。它包含两个构造函数：cons（连接头元素和尾列表）和 nil（空列表）。

```
List=Datatype('List')
List.declare('cons', ('car', IntSort()), ('cdr', List))
List.declare('nil')
List=List.create()
#使用定义好的 List
l1=List.cons(10, List.nil)
print("创建的列表结构:", l1)
```

2. 枚举与树

```
def datatype_experiment():
    print("=== 1. 自定义递归列表 (IntList) ===")

    List = Datatype('IntList')

    List.declare( name: 'cons', *args: ('head', IntSort()), ('tail', List))
    List.declare('nil')
    List = List.create()
    l1 = List.cons(10, List.cons(20, List.nil))
    print("构造的列表结构:")
    print(l1)
    print("\n使用访问器获取数据:")
    head_val = simplify(List.head(l1))
    tail_val = simplify(List.tail(l1))
    print(f"头元素 (head): {head_val}")
    print(f"尾列表 (tail): {tail_val}")
    print("\n=== 2. 自定义枚举类型 (Color) ===")
    Color = Datatype('Color')
    Color.declare('Red')
    Color.declare('Green')
    Color.declare('Blue')
    Color = Color.create()
    c1 = Color.Red
    c2 = Color.Green
    print(f"枚举比较 (Red == Green?): {simplify(c1 == c2)}")
    print(f"枚举比较 (Red != Blue?): {simplify(c1 != Color.Blue)}")
if __name__ == "__main__":
    datatype_experiment()
```

教程还展示了如何定义枚举类型（如 `Color:Red, Green, Blue`）以及二叉树结构。这些高级数据结构极大地扩展了 Z3 的应用场景，使其能够处理更抽象的软件验证任务。

输出：

```
=== 1. 自定义递归列表 (IntList) ===  
构造的列表结构：  
cons(10, cons(20, nil))  
  
使用访问器获取数据：  
头元素 (head): 10  
尾列表 (tail): cons(20, nil)  
  
=== 2. 自定义枚举类型 (Color) ===  
枚举比较 (Red == Green?): False  
枚举比较 (Red != Blue?): True
```

八、量词 (Quantifiers) 与模式匹配

量词是形式化方法中最强大但也最难掌握的部分。Z3 支持全称量词 (ForAll) 和存在量词 (Exists)。

1. 量词建模

我尝试使用量词来定义一个“子类型”关系。如果类型 A 是类型 B 的子类型，且 B 是 C 的子类型，那么 A 也是 C 的子类型（传递性）。

```
def quantifier_experiment():  
    print("=== 量词与模式匹配实验 (Subtype Transitivity) ===")  
    Type = DeclareSort('Type')  
    subtype = Function('subtype', Type, Type, BoolSort())  
    x, y, z = Consts('x y z', Type)  
    axiom = ForAll([x, y, z],  
                    Implies(And(subtype(x, y), subtype(y, z)),  
                             subtype(x, z)),
```

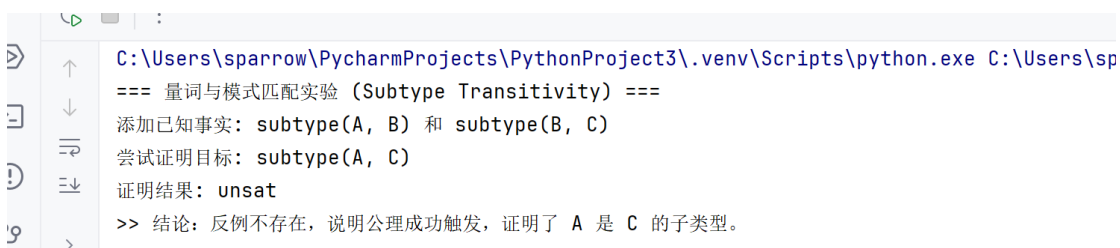
```

        patterns=[MultiPattern(subtype(x, y), subtype(y,
z))])
    s = Solver()
    s.add(axiom)
    A, B, C = Consts('A B C', Type)
    print("添加已知事实: subtype(A, B) 和 subtype(B, C)")
    s.add(subtype(A, B))
    s.add(subtype(B, C))
    print("尝试证明目标: subtype(A, C)")
    s.add(Not(subtype(A, C)))
    result = s.check()
    if result == unsat:
        print(f"证明结果: {result}")
        print(">> 结论: 反例不存在, 说明公理成功触发, 证明了 A 是 C 的子类型。")
    else:
        print(f"证明结果: {result}")
        print(">> 结论: 无法证明。")
if __name__ == "__main__":
    quantifier_experiment()

```

2. 模式 (Patterns) 与 E-matching

教程深入探讨了 Z3 如何处理量词——通过模式匹配 (E-matching)。为了防止量词实例化导致搜索空间爆炸, 我们可以为量词提供“模式 (Pattern)”提示, 告诉求解器何时触发实例化。这是一个高级话题, 教程通过对比有无 Pattern 的求解效率和结果, 展示了其重要性。



```

C:\Users\sparrow\PycharmProjects\PythonProject3\.venv\Scripts\python.exe C:\Users\sp
=== 量词与模式匹配实验 (Subtype Transitivity) ===
添加已知事实: subtype(A, B) 和 subtype(B, C)
尝试证明目标: subtype(A, C)
证明结果: unsat
>> 结论: 反例不存在, 说明公理成功触发, 证明了 A 是 C 的子类型。

```

九、综合应用案例：逻辑谜题解决

为了巩固上述所有知识, 我复现了 Introduction 中提到的几个经典谜题。

1. 数独 (Sudoku) 求解

利用列表推导式和 Distinct 约束，数独问题的建模变得异常简洁。

- 约束 1：每个单元格的值在 1-9 之间。
- 约束 2：每一行包含的数字互不相同 (Distinct)。
- 约束 3：每一列、每一个
3×3 宫格内的数字互不相同。

代码仅用了几十行就描述了整个规则，Z3 在瞬间就填补了缺失的数字。这让我深刻体会到声明式编程“只描述问题，不描述步骤”的魅力。

```
def sudoku_solver():
    print("=== 1. 数独求解 (Sudoku) ===")
    X = [[Int(f"x_{i}_{j}") for j in range(9)] for i in range(9)]
    s = Solver()
    for r in range(9):
        for c in range(9):
            s.add(X[r][c] >= 1, X[r][c] <= 9)
    for r in range(9):
        s.add(Distinct(X[r])) # 每一行
    for c in range(9):
        s.add(Distinct([X[r][c] for r in range(9)])) # 每一列
    for i in range(0, 9, 3):
        for j in range(0, 9, 3):
            cells = [X[r][c] for r in range(i, i + 3) for c in range(j, j + 3)]
            s.add(Distinct(cells))
    instance = [
        [8, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 3, 6, 0, 0, 0, 0, 0],
        [0, 7, 0, 0, 9, 0, 2, 0, 0],
        [0, 5, 0, 0, 0, 7, 0, 0, 0],
        [0, 0, 0, 0, 4, 5, 7, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 3, 0],
        [0, 0, 1, 0, 0, 0, 0, 6, 8],
        [0, 0, 8, 5, 0, 0, 0, 1, 0],
        [0, 9, 0, 0, 0, 0, 4, 0, 0]
    ]
    for r in range(9):
        for c in range(9):
```

```
        if instance[r][c] != 0:
            s.add(X[r][c] == instance[r][c])
    if s.check() == sat:
        m = s.model()
        print("找到解: ")
        for r in range(9):
            row = [str(m.eval(X[r][c])) for c in range(9)]
            print(" ".join(row))
    else:
        print("无解")
```

找到解:

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

2. 安装问题 (Dependency Problem)

教程还介绍了一个软件包管理的依赖问题：安装包 A 依赖包 B，但包 A 与包 C 冲突。通过 Implies (依赖) 和 Not(And(...)) (冲突) 的布尔逻辑组合，Z3 可以自动计算出一个满足所有依赖关系且无冲突的安装方案。

def dependency_problem():
print("\n=== 2. 软件包依赖问题 (Install Problem) ===")
a, b, c, d, e = Bools('a b c d e')
s = Solver()
s.add(Implies(a, b))
s.add(Not(And(a, c)))
s.add(Implies(c, Or(d, e)))
s.push()
s.add(a)
print("尝试安装 a...")
if s.check() == sat:

<code>print("成功, 方案:", s.model())</code>
<code>else:</code>
<code>print("失败")</code>
<code>s.pop()</code>
<code>s.push()</code>
<code>s.add(a, c)</code>
<code>print("尝试同时安装 a 和 c...")</code>
<code>if s.check() == sat:</code>
<code>print("成功")</code>
<code>else:</code>
<code>print("失败 (符合预期, 因为 a 与 c 冲突)")</code>
<code>s.pop()</code>

=== 2. 软件包依赖问题 (Install Problem) ===

尝试安装 a...

成功, 方案: [b = True, a = True, c = False]

尝试同时安装 a 和 c...

失败 (符合预期, 因为 a 与 c 冲突)

十、学习总结与心得

通过对《Introduction》和《AdvancedTopics》两份文档的系统性学习，我从零开始构建了对 SMT 求解器的认知体系。从最基本的整数方程求解，到复杂的位向量分析、数组内存模型以及高阶的量词推理，Z3 展现出了惊人的灵活性和强大的计算能力。

核心收获：

1. 思维范式的转变：从过程式编程转向声明式约束建模。我不再思考“如何通过循环找到解”，而是思考“解必须满足什么性质”。
2. 理论与实践结合：文档中的“位运算技巧证明”和“数独求解”等案例，生动地展示了枯燥的数理逻辑如何转化为解决实际工程问题的代码。
3. 高级特性的掌握：特别是对 Solver 状态管理 (Push/Pop) 和抽象语法树 (AST) 的理解，为将来使用 Z3 进行更复杂的程序分析（如符号执行）打下了坚

实基础。

Z3 不仅仅是一个计算库，它是连接数学逻辑与计算机系统的桥梁。在未来的学习中，我计划进一步探索其在软件安全漏洞挖掘及形式化验证领域的深层应用。